PRODUCT REQUIREMENTS DOCUMENT (PRD)

Life After Code Hackathon — GitLab-Connected Post-Code Engineering Platform

Working Product Name: CodeRescue (provisional)
Document Type: Product Requirements Document
Version: 1.0 — Initial Product Definition
Status: Draft for Review
Project Approach: Start Fresh
Primary Integration: GitLab Duo Agent Platform
Target Users: Advanced developers, beginner developers and amateur software engineers

---

1. Executive Summary

CodeRescue is a third-party, GitLab-connected post-code engineering platform designed to help developers establish, maintain and verify the engineering health of their software projects after implementation.

The platform addresses a common gap in software development: completing the code does not necessarily mean the software is secure, stable, maintainable, testable, deployable or ready for production.

Developers must still address architectural weaknesses, technical debt, security vulnerabilities, privacy requirements, failed pipelines, incomplete testing, deployment readiness and ongoing operational responsibilities. These tasks can be difficult to coordinate, particularly for inexperienced developers or teams inheriting unfamiliar codebases.

CodeRescue will provide two operating modes:

1. **Advanced Developer Mode:** A post-code control center for experienced developers who want to manage engineering tasks, verify the health of their repositories, track CI/CD results and complete outstanding post-code requirements efficiently.
2. **Guided Engineering Mode:** An assisted engineering workflow for beginners and less experienced developers that assesses repository health, establishes a proposed engineering policy baseline, explains the reasoning behind recommendations, stabilizes the system and guides remediation through verification and DevSecOps readiness.

GitLab Duo Agent Platform will serve as the platform's exclusive AI reasoning and agentic orchestration layer. CodeRescue will provide repository intelligence infrastructure, deterministic application logic, policy management, user interfaces, workflow tracking and

evidence aggregation. GitLab and conventional engineering tools will provide repository operations, CI/CD execution and verifiable engineering results.

The platform must ensure that all agent-generated changes are constrained by the applicable development standards, architecture, security requirements and privacy rules.

2. Problem Statement

Software projects frequently reach a point where implementation is considered complete, but essential engineering work remains unfinished or unverified.

Examples include:

- Backend and frontend defects that have not been adequately tested.
- Architectural inconsistencies and tightly coupled components.
- Broken CI/CD pipelines or incomplete automated testing.
- Security weaknesses, exposed secrets and unsafe configurations.
- Dependencies with known vulnerabilities or maintenance risks.
- Inadequate privacy controls and sensitive-data handling.
- Missing deployment safeguards, monitoring and recovery procedures.
- Poor documentation and unclear engineering standards.
- Difficulty prioritizing remediation without introducing further defects.

Existing engineering tools often provide individual findings, test results or code suggestions. Developers still need to interpret those results, understand dependencies between problems, determine the correct order of remediation and verify that changes comply with project requirements.

Beginners face an additional challenge: they may not know which engineering standards their project needs or how to establish them.

CodeRescue aims to close this gap by combining repository intelligence, policy-constrained remediation and a verifiable post-code engineering lifecycle in one platform.

3. Product Vision

To make reliable post-code engineering practices accessible to developers of different experience levels by transforming repository evidence into an understandable, prioritized and verifiable engineering improvement process.

The platform should help developers move from an existing codebase of uncertain quality to a system whose engineering condition, outstanding risks, remediation history and release readiness can be evaluated using documented evidence.

4. Product Objectives

4.1 Primary objectives

1. Connect users to their existing GitLab accounts and authorized repositories.
2. Provide an actionable post-code task management and verification experience for advanced developers.
3. Generate understandable engineering health reports for beginner and amateur developers.
4. Classify engineering findings by domain while recognizing cross-domain dependencies.
5. Establish appropriate engineering policies that constrain every agent-generated change.
6. Use GitLab Duo Agent Platform as the exclusive AI reasoning and agentic orchestration layer.
7. Support controlled stabilization and remediation through authorized, reviewable workflows.
8. Establish and verify appropriate DevSecOps practices.
9. Maintain traceability between findings, policies, remediation tasks, code changes and verification evidence.
10. Demonstrate a complete, working post-code engineering workflow within the hackathon's scope.

4.2 Success criteria

The hackathon MVP should demonstrate that:

- A user can connect an authorized GitLab account and select a repository.
- Repository metadata and available engineering evidence can be collected successfully.
- The platform can present meaningful engineering findings with supporting evidence.
- The platform can produce or manage a policy baseline appropriate to the selected mode.
- An eligible remediation task can be delegated to GitLab Duo through a supported integration.
- The resulting changes can be reviewed and validated using available engineering checks.
- Users can identify unresolved findings and understand the system's remaining risks.

5. Target Users and Personas

5.1 Persona A: Advanced Developer

Profile: An experienced software developer who understands software architecture, engineering standards, security requirements and deployment workflows.

Needs:

- Quickly identify incomplete post-code tasks.
- Inspect actual repository and CI/CD status.
- Define and maintain project-specific engineering policies.
- Delegate appropriate tasks to GitLab Duo.
- Verify changes without unnecessary explanations or workflow restrictions.

- Track release readiness and outstanding risks.

Expected outcome: A concise engineering control center that helps complete and verify post-code work efficiently.

5.2 Persona B: Beginner or Amateur Developer

Profile: A developer who can create or maintain software but has limited experience with architecture governance, DevSecOps, security engineering or production readiness.

Needs:

- Understand the condition of an unfamiliar or existing repository.
- Receive explanations of engineering issues and their impact.
- Understand which problems must be resolved first.
- Obtain a proposed development and security policy baseline.
- Receive explanations for the recommended policies.
- Remediate problems through guided workflows.
- Learn what was changed, why it was changed and how it was verified.

Expected outcome: A safer, more understandable and better-governed software project, with clear evidence of progress and remaining risks.

5.3 Shared user needs

Both personas require:

- Secure GitLab account authorization.
- Accurate repository and pipeline evidence.
- Clear issue prioritization.
- Policy-constrained remediation.
- Transparent execution status.
- Reviewable code changes.
- Verifiable outcomes and an audit trail.

6. Foundational Product Principles

These principles apply to both operating modes and are mandatory architectural constraints.

6.1 Problem-domain awareness

The platform must classify engineering issues into the following domains:

Backend engineering

- API contracts and business logic.
- Database integrity and data access.
- Authentication and authorization.
- Background processing, performance and service boundaries.

Frontend engineering

- User interface behavior and state management.
- API integration and error handling.
- Input validation and accessibility.
- Client-side security and dependency management.

Security and privacy

- Application vulnerabilities.
- Secret exposure and access-control weaknesses.
- Dependency security and configuration risks.
- Sensitive-data handling, privacy controls and applicable obligations.

Operations and delivery

- CI/CD pipeline failures.
- Build, test and deployment configuration.
- Environment consistency.
- Observability, recovery and operational readiness.

A finding may belong to multiple domains. The platform must support cross-domain classifications, relationships between findings, root-cause identification and remediation dependencies.

For example, an API exposing personal information may constitute a backend defect, a security vulnerability and a privacy risk. These must be represented as related aspects of the same underlying problem where appropriate.

The platform must distinguish verified findings from inferred risks and assumptions.

6.2 Policy-constrained remediation

Every agent-generated change must comply with the applicable project policies.

The policy baseline must cover, where relevant:

- Development standards and coding conventions.
- Architecture and design constraints.
- Security requirements and secure development practices.
- Data protection and privacy requirements.
- Testing and quality requirements.
- CI/CD, code review and deployment controls.

No change should be considered compliant merely because it compiles or passes a single test.

Where requirements conflict, are incomplete or cannot be verified, the platform must flag the conflict or uncertainty rather than silently inventing a resolution.

6.3 GitLab Duo exclusivity

GitLab Duo Agent Platform must be the sole AI reasoning and agentic orchestration layer.

The platform must not introduce another LLM provider, independent AI coding agent or competing AI orchestrator to make engineering decisions.

Conventional application logic and non-AI tools may perform deterministic functions, including:

- Repository metadata collection.
- Policy enforcement and workflow state management.
- Test execution and result aggregation.
- Static analysis and security scanning.
- CI/CD monitoring.
- Evidence validation and audit logging.

These tools provide evidence and controls; they do not replace GitLab Duo as the AI reasoning and orchestration layer.

6.4 Evidence-based engineering

The platform must distinguish between:

- What has been observed.
- What has been inferred.
- What has been recommended.
- What has been changed.
- What has been verified.
- What remains uncertain or unresolved.

The platform must never report successful remediation, security compliance or release readiness without sufficient supporting evidence.

7. Product Scope and Operating Modes

7.1 Mode A: Advanced Developer — Post-Code Control Center

Functional requirements

A-01: GitLab account connection

Users must be able to authorize CodeRescue to access their GitLab account through a supported authentication mechanism.

Acceptance criteria:

- The user can initiate and complete authorization.
- The user can select repositories they are authorized to access.
- The platform respects available permissions and reports inaccessible resources.
- The user can revoke the integration.

A-02: GitLab subscription and capability visibility

The platform must provide access to relevant GitLab subscription or entitlement information where supported, along with links to GitLab's official subscription management interfaces.

Acceptance criteria:

- The platform checks relevant Duo capability availability where supported.
- Unsupported or unavailable capabilities are clearly identified.
- The platform does not claim to administer subscriptions unless an explicitly supported and authorized operation is available.

A-03: Post-code checklist

The platform must display an actionable checklist of engineering tasks and verification requirements.

Checklist categories should include:

- Functional correctness.
- Testing and code quality.
- Security and privacy.
- Architecture and maintainability.
- CI/CD and build health.
- Deployment readiness.
- Operational readiness.
- Documentation and handover.

Acceptance criteria:

- Users can inspect task status, evidence and outstanding requirements.
- Completed tasks retain verification evidence.
- Tasks can be prioritized and tracked.
- Healthy projects are not forced through unnecessary remediation.

A-04: Engineering policy configuration

Advanced users must be able to define, import or approve project development standards, architecture constraints, security requirements and privacy rules.

Acceptance criteria:

- Policies can be associated with a repository or relevant component.
- Policy changes are versioned.
- Conflicting or incomplete policies are flagged.
- Remediation tasks reference the applicable policy baseline.

A-05: Verification and workflow execution

Users must be able to initiate supported GitLab Duo workflows for eligible engineering tasks.

Acceptance criteria:

- The user can inspect the intended task before execution.
- Workflow execution status is visible.
- Proposed changes can be reviewed.
- Available tests and CI/CD checks can be inspected.
- Failed or incomplete verification is clearly distinguished from success.

A-06: Release readiness

The platform must provide an evidence-based view of remaining engineering work and release blockers.

Acceptance criteria:

- Critical unresolved findings are visible.
- Failed mandatory checks prevent the platform from declaring readiness.
- Exceptions are recorded explicitly.
- The platform reports a readiness assessment rather than making unsupported guarantees.

7.2 Mode B: Guided Engineering — Health, Stabilization and Remediation

Functional requirements

B-01: Repository discovery

The platform must collect available repository metadata, project structure, configuration information, dependency data, CI/CD results and relevant code evidence within authorized access boundaries.

Acceptance criteria:

- The platform identifies the evidence successfully collected.
- Missing permissions and unavailable evidence are reported.
- The platform does not infer that an uninspected component is healthy.

B-02: Engineering health report

GitLab Duo Agent Platform must be used to interpret available evidence and produce an understandable engineering health report.

The report should include:

- Project and technology overview.
- Backend, frontend, security/privacy and operations findings.
- Cross-domain relationships and likely root causes.
- Severity, impact and supporting evidence.
- Architectural and maintainability concerns.
- Test and CI/CD health.

- Deployment and operational readiness.
- Uncertainties, missing evidence and recommended next steps.

Acceptance criteria:

- Findings are linked to available supporting evidence.
- Each finding has an understandable explanation.
- Recommendations distinguish confirmed defects from potential risks.
- Cross-domain findings are represented without unnecessary duplication.

B-03: Orchestrator-defined engineering policy baseline

GitLab Duo Agent Platform must propose a project-specific engineering policy baseline using the repository evidence and user-provided context.

The baseline should address:

- Development standards.
- Architecture and design constraints.
- Security requirements.
- Privacy and data protection requirements.
- Testing and quality gates.
- CI/CD and deployment safeguards.

Acceptance criteria:

- The report explains why each significant policy is recommended.
- Recommendations identify their supporting evidence, assumptions and trade-offs.
- Existing project conventions are preserved unless a justified change is proposed.
- Legal and regulatory obligations are not claimed as satisfied solely from code inspection.
- Policies requiring clarification are marked accordingly.
- The proposed baseline is presented for user review and approval before becoming binding remediation policy.

B-04: Stabilization planning

The platform must produce a sequenced plan to address foundational weaknesses before undertaking dependent remediation.

Potential stabilization activities include:

- Restoring failed builds and pipelines.
- Establishing reliable test execution.
- Resolving critical configuration issues.
- Addressing architectural blockers.
- Establishing baseline security controls.
- Resolving dependency and environment inconsistencies.

Acceptance criteria:

- Tasks have priorities and dependencies.
- The plan explains why foundational tasks precede dependent tasks.
- The platform distinguishes blockers from improvements that can be deferred.
- The plan respects the approved engineering policy baseline.

B-05: Guided remediation

The platform must support incremental, reviewable remediation through authorized GitLab Duo workflows.

Acceptance criteria:

- Each task references its originating finding and applicable policy version.
- The proposed action and relevant risks are presented to the user.
- Approval requirements are enforced before protected actions.
- Changes are made through supported, auditable GitLab operations.
- Validation failures are reported and linked to the affected task.
- The platform does not mark a finding resolved without sufficient verification evidence.

B-06: DevSecOps establishment

The platform must guide the user in establishing appropriate engineering and delivery controls.

These may include:

- Automated testing and quality gates.
- Static application security testing.
- Secret detection.
- Dependency vulnerability scanning.
- Dynamic application security testing where appropriate.
- Protected branches and merge-request review.
- Deployment approvals and environment separation.
- Configuration and secret-management practices.
- Logging, monitoring and recovery readiness.

Acceptance criteria:

- The platform identifies which controls already exist.
- Missing controls are prioritized based on the project's risk and context.
- Recommendations are accompanied by explanations.
- Verification status is based on actual configuration or execution evidence.
- Controls are not represented as active merely because they have been recommended.

B-07: Learning and engineering explanations

The platform must explain important findings, policy decisions and remediation outcomes in language appropriate for less experienced developers.

Acceptance criteria:

- Explanations describe the problem, its impact and the proposed resolution.
- Users can understand why a task has a particular priority.
- Verification results explain what passed, failed or remains untested.
- Uncertainty and unresolved risks are clearly communicated.

8. Shared Engineering Policy Model

Both modes must use a common policy-management mechanism.

Each policy record should support the following information:

- Policy identifier and title.
- Policy category.
- Requirement and rationale.
- Source of the policy.
- Applicable repository, component or engineering domain.
- Validation method.
- Severity of non-compliance.
- Version and effective status.
- Approval status.
- Exceptions and their authorization.
- Evidence used to evaluate compliance.

Policy ownership

In Mode A, the developer defines or approves the policies.

In Mode B, GitLab Duo proposes the policies and explains its reasoning. The user reviews and approves the resulting baseline.

Policy enforcement

Every remediation task must reference the relevant policy version.

Before a proposed change is accepted as complete, the platform must evaluate available evidence against the applicable requirements.

When a policy changes, affected outstanding tasks must be identified for reassessment where necessary.

9. Shared Post-Code Lifecycle

The platform must support the following lifecycle in both modes:

1. **Connect and discover:** Authorize repository access and gather available evidence.
2. **Assess engineering health:** Identify defects, risks, architectural concerns and incomplete engineering tasks.

3. **Establish the policy baseline:** Apply developer-defined policies in Mode A or propose a policy baseline in Mode B.
4. **Stabilize:** Address foundational weaknesses and blockers in a safe sequence.
5. **Remediate:** Delegate eligible tasks to GitLab Duo and manage the resulting changes.
6. **Establish DevSecOps:** Configure and verify appropriate quality, security and delivery controls.
7. **Verify and release:** Run relevant checks, inspect evidence and evaluate release readiness.
8. **Monitor and maintain:** Track subsequent failures, regressions, dependency risks and outstanding work.

The lifecycle must be adaptive. A healthy repository can proceed directly to verification, while a fragile repository may require several stabilization stages.

10. System Responsibilities and Integration Boundaries

10.1 CodeRescue responsibilities

The application must provide:

- User authentication and integration management.
- Repository selection and metadata collection.
- Engineering evidence aggregation.
- Structured finding and policy storage.
- Domain classification and relationship tracking.
- Task prioritization and lifecycle management.
- User interfaces for both operating modes.
- Approval workflows and permission-aware actions.
- Audit logging and progress reporting.
- Verification evidence and release-readiness reporting.

CodeRescue may perform deterministic analysis and enforce rules, but it must not introduce a separate AI reasoning or orchestration system.

10.2 GitLab Duo Agent Platform responsibilities

GitLab Duo must provide the AI-driven reasoning and agentic execution supported by the available integration.

Potential delegated activities include:

- Interpreting repository context.
- Analyzing engineering findings.
- Proposing engineering policies and remediation plans.
- Investigating selected defects.
- Producing or modifying code through supported workflows.
- Preparing changes for review.
- Supporting eligible CI/CD and engineering tasks.

The actual supported actions depend on the available Duo APIs, flows, project configuration, permissions and subscription entitlements. The implementation must verify these prerequisites.

10.3 GitLab and conventional engineering tools

GitLab and conventional tools provide:

- Repository and branch management.
- Merge requests and code review.
- CI/CD pipeline execution.
- Test results and build evidence.
- Available security scanning and dependency information.
- Permission enforcement.
- Execution status and audit information.

The platform must not confuse an agent's statement that a task is complete with independently verifiable engineering evidence.

11. Non-Functional Requirements

11.1 Security

- Apply least-privilege access to GitLab resources.
- Store credentials and tokens securely.
- Never expose access tokens in client-visible responses or logs.
- Support authorization revocation and appropriate token invalidation.
- Require authorization for sensitive repository operations.
- Protect against prompt injection and malicious instructions embedded in repository content.
- Treat repository files, issues, comments and other retrieved content as untrusted input.
- Prevent unauthorized cross-project data access.
- Maintain auditable records of security-relevant actions.

11.2 Privacy and data protection

- Collect only the repository information required for the selected workflow.
- Disclose what information is collected, why it is collected and how it is used.
- Define retention and deletion requirements.
- Avoid unnecessary storage of source code and sensitive repository contents.
- Respect repository permissions and applicable data protection obligations.
- Establish the relevant privacy requirements based on the project context and applicable jurisdiction.
- Document the data flows between CodeRescue, GitLab and its authorized integrations.

11.3 Reliability and integrity

- Handle API failures, rate limits and interrupted workflows gracefully.

- Preserve workflow state and task history.
- Distinguish pending, running, failed, cancelled and completed executions.
- Avoid duplicate execution of non-idempotent operations where possible.
- Reconcile displayed results with authoritative GitLab status.
- Never fabricate verification results.

11.4 Usability and accessibility

- Provide distinct experiences for advanced and beginner users.
- Use clear status indicators and actionable explanations.
- Ensure important findings are understandable without requiring users to inspect raw logs.
- Support accessible navigation, readable layouts and useful error messages.

11.5 Maintainability

- Separate GitLab integration, policy management, evidence collection, workflow tracking and user-interface responsibilities.
- Maintain documented interfaces and data models.
- Support automated testing and repeatable deployment.
- Keep the application extensible without introducing competing AI orchestration layers.

12. Data Requirements

The application should maintain the following logical records:

1. **User:** Identity, preferences and application-level permissions.
2. **GitLab Connection:** Authorized account reference, scopes, connection status and revocation state.
3. **Repository:** Project identifier, metadata, access status and integration settings.
4. **Engineering Assessment:** Assessment timestamp, scope, evidence coverage and overall findings.
5. **Finding:** Description, domains, severity, evidence, relationships and resolution status.
6. **Engineering Policy:** Requirement, source, rationale, version, applicability and approval status.
7. **Remediation Task:** Linked finding, policy version, priority, dependencies and execution status.
8. **Agent Workflow:** Supported Duo workflow reference, project context, status and execution metadata.
9. **Verification Result:** Test or check performed, result, timestamp and supporting evidence.
10. **Audit Event:** Actor, action, target resource, timestamp and outcome.

Sensitive values, credentials and source-code content must not be stored in these records unless explicitly required and appropriately protected.

13. User Journey Requirements

13.1 Advanced Developer journey

1. The developer signs in to CodeRescue.
2. The developer connects a GitLab account.
3. The developer selects a repository.
4. The platform collects available engineering evidence.
5. The developer reviews the post-code checklist and verification results.
6. The developer defines or confirms the policy baseline.
7. The developer selects an outstanding task.
8. The platform delegates the eligible task to a supported Duo workflow.
9. The developer reviews the proposed changes and validation evidence.
10. The platform updates task status and release readiness.

13.2 Guided Engineering journey

1. The developer signs in and connects a GitLab repository.
2. The platform discovers available project metadata and engineering evidence.
3. GitLab Duo analyzes the available evidence.
4. The user receives an engineering health report.
5. GitLab Duo proposes an engineering policy baseline with explanations.
6. The user reviews and approves the applicable policies.
7. The platform presents a sequenced stabilization plan.
8. The user authorizes remediation tasks as required.
9. GitLab Duo executes eligible tasks through supported workflows.
10. Automated checks and reviewable changes provide verification evidence.
11. The user receives an updated engineering health report and DevSecOps readiness assessment.

14. MVP Scope

The hackathon MVP must prioritize a complete, demonstrable engineering workflow over a large number of partially implemented features.

In scope

- New CodeRescue application and public project repository.
- GitLab account authorization and repository selection.
- Repository metadata and available CI/CD evidence collection.
- Two operating modes with differentiated interfaces.
- Structured engineering findings and cross-domain classification.
- Advanced Developer policy configuration.
- Guided Engineering policy recommendations and explanations generated through Duo.
- An actionable post-code checklist.
- A sequenced stabilization plan.
- At least one working Duo-powered remediation workflow.
- Reviewable changes and available automated verification.
- A basic DevSecOps readiness report.
- Workflow status, audit history and unresolved-risk reporting.

- Automated tests and a reproducible deployment process.
- Hackathon documentation and demonstration.

Out of scope for the initial MVP

- Guaranteed autonomous remediation of every possible engineering issue.
- Support for every programming language, framework and GitLab configuration.
- Full subscription purchasing or billing administration.
- Complete enterprise compliance certification.
- Guaranteed legal compliance based solely on automated repository analysis.
- Unrestricted production deployment without authorization.
- Replacing GitLab's native repository, CI/CD or security infrastructure.
- Supporting alternative AI orchestrators.
- Fully automated remediation of every cross-domain architectural problem.
- Comprehensive production monitoring across arbitrary external infrastructure.

These capabilities may be considered in future product iterations after the core workflow is demonstrated.

15. Acceptance Criteria for the Hackathon Demonstration

The MVP will be considered functionally demonstrated when the following end-to-end scenario succeeds.

Stage 1: Repository connection

- A user authorizes the application to access a GitLab repository.
- The application displays repository information and available evidence.
- Unsupported capabilities and access limitations are reported clearly.

Stage 2: Engineering assessment

- The platform identifies at least one meaningful engineering issue.
- The issue is classified into the relevant domain or domains.
- The report provides supporting evidence and explains the issue's impact.
- Findings are distinguished from assumptions and missing evidence.

Stage 3: Policy baseline

- Mode A can apply a developer-defined policy.
- Mode B can generate an explanatory proposed policy baseline through GitLab Duo.
- The applicable policy baseline is recorded and associated with remediation tasks.

Stage 4: Remediation

- A supported GitLab Duo workflow is triggered for a selected task.
- The execution status is tracked.
- The resulting change is available for review through a supported GitLab workflow.
- Applicable approval requirements are enforced.

Stage 5: Verification

- Relevant automated checks execute or their authoritative results are retrieved.
- The platform distinguishes successful checks from failures and unavailable evidence.
- The finding's status reflects the verification outcome.
- Remaining risks and incomplete tasks remain visible.

Stage 6: Final report

- The user can inspect the initial finding, applicable policy, remediation history and verification evidence.
- The final report explains what improved and what remains unresolved.
- The application does not claim that the entire repository is secure or production-ready based on one successful remediation.

16. Success Metrics

The following are proposed MVP targets, to be validated during implementation.

Metric	Proposed target
GitLab connection	Successful authorization and repository selection in the demo environment
Assessment traceability	All displayed findings linked to available evidence or explicitly marked as inferred
Domain classification	All demonstrated findings assigned relevant domain labels
Policy traceability	Every demonstrated remediation task linked to an applicable policy version
Duo integration	At least one successful end-to-end agentic workflow
Verification integrity	No task reported as successfully remediated without sufficient verification evidence
Auditability	The demonstrated workflow can be traced from finding to verification
User comprehension	A beginner can explain the purpose of the demonstrated finding and remediation after reviewing the report
CI/CD integration	At least one relevant automated validation result is visible in the demonstrated workflow

These are proposed product acceptance targets, not claims of existing functionality.

17. Risks and Mitigation Strategies

17.1 GitLab Duo integration constraints

Risk: Required APIs, flows or capabilities may be unavailable under the selected account or subscription.

Mitigation: Validate authentication, entitlement, permissions and a real workflow early. Select a supported remediation scenario before building dependent features.

17.2 Incorrect or incomplete repository assessment

Risk: Limited repository access or missing evidence may lead to inaccurate conclusions.

Mitigation: Record evidence coverage, flag uncertainty and avoid presenting inferred findings as confirmed defects.

17.3 Unsafe remediation

Risk: Agent-generated changes may break existing functionality or violate project constraints.

Mitigation: Enforce policy constraints, use reviewable changes, run relevant checks and require approval for protected actions.

17.4 Conflicting engineering policies

Risk: Existing architecture, developer preferences and security requirements may conflict.

Mitigation: Version policies, identify conflicts explicitly and require authorized resolution rather than silently overriding constraints.

17.5 Excessive MVP scope

Risk: Attempting to support every engineering domain and remediation workflow may prevent completion of the core product.

Mitigation: Implement one complete workflow first, then expand the breadth of findings and controls.

17.6 Misleading compliance or readiness claims

Risk: Users may interpret an automated report as proof of complete security or legal compliance.

Mitigation: Report evidence, scope, limitations, unresolved risks and verification coverage. Avoid unsupported guarantees.

17.7 Repository prompt injection

Risk: Malicious instructions in source files, comments or documentation could influence agent behavior.

Mitigation: Treat repository content as untrusted data, enforce tool permissions and policy constraints outside the agent's discretion, and require review for sensitive operations.

18. Dependencies and Assumptions

The project depends on:

- A GitLab account and repository available for development and demonstration.
- An eligible GitLab Duo Agent Platform configuration.
- Supported authentication and workflow APIs.
- Permissions to read repository information and initiate the selected workflow.
- A sample repository containing a reproducible engineering issue.
- An available CI/CD pipeline or equivalent validation mechanism.
- A public GitLab project repository and hackathon submission assets.

The following assumptions must be validated before implementation is finalized:

1. The chosen GitLab environment permits the required Duo workflow integration.
2. Repository metadata and CI/CD evidence can be collected within the authorized access model.
3. The selected remediation can be completed using a supported Duo workflow.
4. The hackathon's current rules permit the proposed architecture and demonstration approach.
5. The available implementation time supports both mode interfaces and at least one complete remediation workflow.

19. Implementation Priorities

Priority 0 — Technical feasibility

- Verify hackathon eligibility and submission requirements.
- Validate GitLab authorization.
- Confirm Duo entitlement and workflow availability.
- Trigger and observe a real supported Duo workflow.
- Identify a reproducible repository issue.

Priority 1 — Core engineering workflow

- Implement repository onboarding.
- Collect metadata and CI/CD evidence.
- Establish the structured findings and policy data models.
- Implement the initial health report.
- Implement the shared task and workflow tracking mechanism.

Priority 2 — Two operating modes

- Build the advanced post-code checklist.
- Implement developer-defined policies.
- Build the guided engineering report.
- Implement Duo-generated policy recommendations and explanations.

Priority 3 — Stabilization and verification

- Implement prioritized remediation planning.
- Integrate the selected Duo remediation workflow.
- Present reviewable changes and verification results.
- Generate the final engineering readiness report.

Priority 4 — Submission readiness

- Complete automated tests and error handling.
- Review access controls and security.
- Document setup and supported capabilities.
- Prepare the demonstration repository.
- Record the hackathon demonstration.
- Verify the public repository, license and submission requirements.

20. Definition of Done

The hackathon MVP is complete when:

- The application can connect to an authorized GitLab repository.
- Both operating modes are represented by functional, differentiated user experiences.
- The system demonstrates problem-domain awareness.
- The system supports policy-constrained remediation.
- GitLab Duo Agent Platform performs the AI-driven reasoning and delegated workflow in the demonstration.
- At least one remediation task is completed or attempted with its actual outcome accurately reported.
- The application presents reviewable changes and relevant verification evidence.
- The initial and final engineering states can be compared.
- Unresolved findings, limitations and outstanding risks remain visible.
- The application passes its required automated tests.
- The public project documentation explains architecture, setup, supported capabilities and limitations.
- Hackathon submission requirements are verified and satisfied.

21. Future Product Direction

After the hackathon, potential extensions include:

- Broader language and framework coverage.
- More comprehensive architectural dependency mapping.
- Additional supported Duo workflows for remediation and CI/CD repair.
- Policy templates for different application types and jurisdictions.
- Longitudinal engineering health tracking.
- Regression detection and recurring post-code assessments.
- Team-level governance and approval workflows.
- More detailed DevSecOps maturity assessments.
- Integration with operational telemetry where authorized.

- Improved educational explanations and guided engineering experiences.

These extensions must retain the foundational requirements: GitLab Duo exclusivity for AI orchestration, problem-domain awareness, policy-constrained remediation, evidence-based verification and appropriate user authorization.

22. Final Product Definition

CodeRescue is a GitLab-connected post-code engineering platform that helps developers assess, stabilize, remediate and verify software systems through two operating modes.

Advanced developers retain control over their engineering policies and use the platform to manage post-code tasks efficiently. Beginners receive a guided engineering assessment and an explained, proposed policy baseline that supports safer remediation and DevSecOps establishment.

Both modes use the same underlying engineering intelligence infrastructure and follow the same policy-constrained lifecycle. GitLab Duo Agent Platform is the exclusive AI reasoning and agentic orchestration layer, while CodeRescue manages the application experience, evidence, policies, task coordination and auditability.

The platform's central promise is not that AI can automatically fix every problem. It is that engineering work after implementation can become **understandable, prioritized, policy-governed and verifiably completed**.